Small-C

For COS

First Edition (October 1981)

Changes are made periodically to the information herein; any such changes will be reported in subsequent revisions.

This manual describes Small-C Version 1.00, the C compiler for the COS disk operating system on the pc86 personal computer.

Small-C is supplied on the COS System Diskette as the program `SMALLC.EXE`, with the library `CLIB.LIB`, the header files (`STDIO.H` and seven others) and the sample program `HELLO.C`. Read Chapter 2, "Writing and Running Programs," before you write your first program.

Preface

This manual tells you how to write programs in C for the pc86 personal computer, and how to compile and run them under COS with Small-C, the C compiler of COS. Small-C is a compact C: it has the integer types, pointers, arrays, structures, unions, enumerations, the preprocessor and a library of functions for files, strings and memory, and it makes small, fast programs that can reach every part of the machine — its ports, its memory, its interrupts, and assembly language written inside the C program itself.

This is not a tutorial on the C language. It assumes that you know C, or are learning it from a textbook, and that you know how to use COS: how to start it, how to name files and how to use the text editor, EDIT. These are described in the COS User's Guide. What this manual describes is the C that Small-C accepts, what it leaves out, and what is special to the pc86.

- If you are new to Small-C, read Part One, Chapters 1 to 4, in order, and try the examples on the computer.
- If you want to use the machine itself — the keyboard, the speaker, the screen — read Chapter 5. For programs of several files and libraries of your own, read Chapter 6.
- Part Two is for reference: the library functions, the error messages, and the technical details of the code Small-C makes.

Organization of This Manual

`Chapter 1`	Introduction: what Small-C is, what you need, and a first program.
`Chapter 2`	Writing and Running Programs: EDIT and Small-C, the options, mistakes, programs of several files, .COM and .EXE programs.
`Chapter 3`	The Language: types, declarations, expressions, functions, statements, and what Small-C does not have.
`Chapter 4`	The Preprocessor: `#include`, `#define`, conditional compilation and `#asm`.
`Chapter 5`	Using the Machine: ports, memory, interrupts, the keyboard and screen, assembly language, and running other programs.
`Chapter 6`	Modules and Libraries: how programs are put together, and how to make libraries.
`Chapter 7`	Library Reference: every function of the library, by header file.
`Appendix A`	Error Messages.
`Appendix B`	Technical Information: how Small-C makes code, memory, limits.
`Appendix C`	The Sample Program `WORDS.C`.

Notation

italics	Items you supply: a file name, a number, an expression.
`[ ]`	The item inside is optional. Do not type the brackets.
`...`	The item before may be repeated.
`smallc hello.c`	In the screen examples, what you type is printed in bold and underlined. Press **Enter** at the end of each line.
`0x1F`	A hexadecimal number, as it is written in C. Other manuals of the pc86 write 1Fh.
`A:\>`	The COS prompt. The programs in this manual were typed in small letters; COS does not mind.

The screen examples in this manual were made on the machine itself.

Contents

PART ONE: USING SMALL-C

Chapter 1. Introduction **1-1**
What Small-C Is 1-2
What You Need 1-2
A First Program 1-2
What Small-C Does Not Have 1-3

Chapter 2. Writing and Running Programs **2-1**
Writing a Program 2-2
Running Small-C 2-2
The Options 2-3
When the Program Has a Mistake 2-4
Messages From the Link 2-5
Programs of Several Files 2-5
.COM and .EXE Programs 2-7
Where Small-C Finds Its Files 2-8
How Small-C Makes a Program 2-8
The Command Line of a Program 2-9

Chapter 3. The Language **3-1**
What Small-C Accepts 3-2
Types and Constants 3-2
Declarations 3-4
Expressions 3-4
Arrays and Pointers 3-5
Structures, Unions and Enumerations 3-6
Functions 3-7
Statements 3-9
What Small-C Does Not Have 3-10

Chapter 4. The Preprocessor **4-1**
How the Preprocessor Works 4-2
Macros 4-2
Conditions 4-3
Predefined Names 4-3

Chapter 5. Using the Machine **5-1**
What a C Program Can Reach 5-2

Ports 5-2
Memory 5-3
Interrupts 5-4
The Keyboard and the Screen 5-5
Assembly Language in C 5-5
Running Other Programs 5-6
Memory for Data 5-7

Chapter 6. Modules and Libraries 6-1
Modules 6-2
Making a Library 6-2
Using a Library 6-2
Rules 6-3
Modules in Assembly Language 6-3
Start-up Modules 6-4

PART TWO: REFERENCE

Chapter 7. Library Reference 7-1
How to Read This Chapter 7-2
STDIO.H 7-2
STDLIB.H 7-4
STRING.H 7-5
CTYPE.H 7-6
STDARG.H 7-6
IO.H 7-7
DOS.H 7-8
CONIO.H 7-8

APPENDIXES

Appendix A. Error Messages A-1
How Messages Look A-2
Messages About the Command Line and the Link A-2
Messages About Files and Lines A-3
Messages About the Preprocessor A-4
Messages About Declarations A-5
Messages About Expressions A-7
Messages About Statements A-8

Appendix B. Technical Information B-1

The Calling Convention B-2
Names and Labels B-2
Data B-3
Memory of a .COM Program B-3
Memory of an .EXE Program B-3
Limits of the Compiler B-4
Keywords B-4
A Module B-5

Appendix C. The Sample Program WORDS.C C-1
Counting Words C-2
The Listing C-2

Quick Reference

This page intentionally left blank.

1 Introduction

Contents

What Small-C Is 1-2
What You Need 1-2
A First Program 1-2
What Small-C Does Not Have 1-3

1

What Small-C Is

Small-C is a compiler for the C programming language. You write a program as a text file, with the COS editor, EDIT, in a file whose name ends in .C; Small-C translates it into a program of the machine's own language, a file ending in .COM, that you run by typing its name, as you run any other COS program.

C was made for writing programs that are close to the machine: operating systems, editors, compilers, games. Small-C is a compiler of that kind. It has no floating point and no `long` integers, but it makes compact code, and a C program can do everything an assembly language program can: read and write the ports of the computer, look at any place in memory, call any COS or BIOS service, and even contain instructions of the 8086 processor between its own lines.

Small-C is itself a C program. It is written in the C that it compiles, and it compiles itself.

What You Need

- A pc86 personal computer with 640 KB of memory and the COS disk operating system.
- The COS System Diskette. These files on it are for Small-C: `SMALLC.EXE`, the compiler; `CLIB.LIB`, the library; the header files `CONIO.H`, `CTYPE.H`, `DOS.H`, `IO.H`, `STDARG.H`, `STDIO.H`, `STDLIB.H` and `STRING.H`; and the sample program `HELLO.C`. Small-C also needs `ASM.COM`, the assembler, and `EDIT.COM`, the editor, that are on the same diskette.
- Room on the diskette for what Small-C writes: a file of assembly language, `CCTEMP.ASM`, that is larger than the program (about 32 KB for a small one), and the program itself.
- Patience. Small-C does a great deal of work for a computer with an 8086 processor: to compile the sample program takes about fifty seconds (Chapter 2 tells where the time goes).

A First Program

The System Diskette has the smallest C program, `HELLO.C`. This is what it looks like:

```
/* hello.c - the first C program: SMALLC HELLO.C, then HELLO */
#include <stdio.h>

int main(void)
{
    printf("Hello from the 8086, in C!\n");
    return 0;
}
```

The first line is a comment. The second asks for `STDIO.H`, the file that tells Small-C about `printf`, the function that shows text on the screen. Every C program has a function called *`main`*, where it starts; this one has a single statement, that calls `printf`, and ends with return 0, which tells COS that the program worked.

To compile and run it:

```
A:\>smallc hello.c
HELLO.COM: 3988 bytes

A:\>hello
Hello from the 8086, in C!

A:\>
```

Small-C shows the name of the program it made and its size. `HELLO.COM` is now on the diskette; HELLO runs it.

What Small-C Does Not Have

Small-C is C without some of its more advanced parts. It has no *`long`*, *`float`* or *`double`* types (integers are 16 bits, and there is no arithmetic with fractions), no *`typedef`*, no pointers to functions, no bit fields, and a structure cannot be passed to a function or returned from it by value. Chapter 3 lists what is missing and what to do instead; each of these things gives an error message when a program uses it, so nothing is silently wrong. Everything else of the language that a program for this computer needs is there.

This page intentionally left blank.

2

Writing and Running Programs

Contents

Writing a Program 2-2
Running Small-C 2-2
The Options 2-3
When the Program Has a Mistake 2-4
Messages From the Link 2-5
Programs of Several Files 2-5
.COM and .EXE Programs 2-7
Where Small-C Finds Its Files 2-8
How Small-C Makes a Program 2-8
The Command Line of a Program 2-9

Writing a Program

A C program is an ordinary text file, written with EDIT. Give it a name that ends in .C, and give the whole name, extension included:

```
A:\>edit squares.c
```

EDIT starts with an empty screen and the message "New file", and the file is made when you save it. Type the program a line at a time, pressing **Enter** at the end of each line, and indent lines with spaces to show their structure: Small-C ignores them. Save the program with **F2** and leave EDIT with **Esc**. This is the screen with the program of this chapter:

```
 COS Edit  SQUARES.C                        Line 1 of 11      Col 1     Insert
/* SQUARES.C - the squares of the numbers 1 to 5 */
#include <stdio.h>

int main(void)
{
    int i;

    for (i = 1; i <= 5; i++)
        printf("%d squared is %d\n", i, i * i);
    return 0;
}




 F1 Help  F2 Save  F3 Find  F4 Again  F5 Line  Ctrl+Y Cut  F9 Paste  Esc Exit
```

Chapter 9 of the COS User's Guide describes EDIT.

Running Small-C

Format

SMALLC [*options*] *file ...*

Small-C takes one or more files, and the options that tell it what to make of them. The file names must have their extensions, because the extension says what the file is:

File	What Small-C does with it
`name.C`	A C source. It is compiled.
`name.ASM`	A module in assembly language. It is taken as it is.
`name.LIB`	A library. The program takes from it the modules that it needs, and no others (Chapter 6).

A file named without an extension is taken for a .C file: SMALLC HELLO is the same as SMALLC `HELLO.C`. Up to 24 files may be given.

Without options, Small-C makes a .COM program, with the name of the first file:

```
A:\>smallc squares.c
SQUARES.COM: 4014 bytes

A:\>squares
1 squared is 1
2 squared is 4
3 squared is 9
4 squared is 16
5 squared is 25

A:\>
```

Typing SMALLC alone shows the format and the options:

```
A:\>smallc
Small-C for COS Version 1.00
usage: SMALLC [options] file...
  file.C    compiled       file.ASM  taken whole
  file.LIB  modules taken as the program needs them
  -o name   the program (NAME.COM after the first file)
  -e        an .EXE: 64 KB of code and 64 KB of data
  -c        each .C to an .ASM module; with -o all in one library
  -S        only the program's .ASM, without ASM
  -Dname[=value]  -Idirectory  -Llibrary  -n (no CLIB.LIB)

A:\>
```

The Options

An option begins with a hyphen, or with a slash. Capital and small letters are the same for the letters that name an option, but not in the names that follow -D, -I and -L. Options may be anywhere in the line, among the files.

Option	Meaning
`-o` *name*	The name of the program (or, with -c, of the library). Without it, the program has the name of the first file, with the extension .COM, or .EXE with -e. A name ending in .EXE makes an .EXE, as -e does.
`-e`	Makes an .EXE program: the code has a segment of its own, and so has the data, with room for 64 KB of data, variables, heap and stack (Appendix B).
`-c`	Compiles each .C file to a module in assembly language, with the name of the file and the extension .ASM, and stops there. With -o, all the files are put in one library file, .ASM files too.
`-S`	Makes only the assembly language of the program, a file with the name of the program and the extension .ASM, and does not run ASM.

Option	Meaning
-D*name*[=*value*]	Defines a macro for every file, as #define does. Without a value, the macro is 1. There is no space after -D.
-I*directory*	Adds a directory where #include looks for files (see below). There is no space after -I. Up to three may be given.
-L*library*	Adds a library, a .LIB file, as if it were given among the files, but after all of them. Up to three may be given.
-n	Does not use CLIB.LIB. The program must get its start-up from a library of its own (-L), and its functions too.

When the Program Has a Mistake

Small-C reads all of the program. When a line breaks the rules of the language, Small-C shows the name of the file, the number of the line and what is wrong, and goes on reading, to find more mistakes; it makes no program. This one has two:

```
/* OOPS.C - a program with mistakes */
#include <stdio.h>

int main(void)
{
    int n;

    n = 3
    printf("%d\n", n + total);
    return 0;
}
```

```
A:\>smallc oops.c
oops.c(9): ';' expected
oops.c(9): 'total' is not declared

A:\>
```

The missing semicolon at the end of line 8 is found only when Small-C sees the printf on line 9, so the message says line 9: when a message does not make sense on its line, look at the line before it. The second message is a result of the first, as often happens: Small-C lost its place, and found 'total' on the way, which was never declared. **Correct the first message first**, then compile again, and see which of the others are left.

Small-C stops after 20 messages, with "too many errors". The messages are listed in Appendix A. The name of the file is as you typed it; for a mistake in a header file it is the name of that file, so that you know where to go in EDIT: press F5, type the line number, correct it and save.

When Small-C finds a mistake, it writes no program, and the one made by an earlier compilation stays as it was. Small-C ends with return code 1; when it made the program it ends with 0.

Messages From the Link

A program is made of its own modules and of the library functions that it calls. When the program is all read, Small-C looks for the names that are used and are not defined anywhere:

```
A:\>smallc bad.c
SMALLC: 'twice' is not defined anywhere
SMALLC: 'missing' is not defined anywhere

A:\>
```

These are the names of functions, or of variables, that the program uses and nobody defines: the spelling is wrong, a file is missing from the line, or a library was not given. Names of C are shown as you wrote them; the names of the assembly language that are not C names are shown with their underscore (Chapter 6). A name that is defined twice is reported too: "_twice is defined in two modules" (see "Programs of Several Files").

Programs of Several Files

A large program is better in several files, each with the functions that belong together. Give them all to Small-C; the program has the name of the first:

```
/* VEC.H - sums and maximums of arrays of numbers */
int vsum(int *v, int n);
int vmax(int *v, int n);
extern int vcalls;
```

```
/* VEC.C - sums and maximums of arrays of numbers */
#include "vec.h"

int vcalls;

int vsum(int *v, int n)
{
    int i, s;

    vcalls++;
    s = 0;
    for (i = 0; i < n; i++)
        s += v[i];
    return s;
}
```

```
int vmax(int *v, int n)
{
    int i, m;

    vcalls++;
    m = v[0];
    for (i = 1; i < n; i++)
        if (v[i] > m)
            m = v[i];
    return m;
}
```

```
/* DEMO.C - uses the functions of VEC.C */
#include <stdio.h>
#include "vec.h"

int data[] = { 31, 4, 159, 26, 5, 358 };

int main(void)
{
    printf("sum %d, largest %d\n", vsum(data, 6), vmax(data, 6));
    printf("%d calls\n", vcalls);
    return 0;
}
```

```
A:\>smallc demo.c vec.c
DEMO.COM: 4190 bytes

A:\>demo
sum 583, largest 358
2 calls

A:\>
```

`VEC.H` says what `VEC.C` offers, and both `DEMO.C` and `VEC.C` include it. Three rules make the files work together:

- A function or a variable that other files use is defined in *one* file only. The other files declare it: a function with its prototype, a variable with **`extern`**.
- A name declared **`static`** belongs to its file, and no other file can see it. Two files may each have a `static` function helper, and they are two different functions.
- A variable declared without `extern` in several files, like **`int` counter;** in all of them, is one variable. If it is given a value in one of them, that one defines it; if two give it a value, or two files have a function with the same name, the message is "*name* is defined in two modules". The tidy way is the definition in one file and an `extern` declaration in the header.

Compiling all the files at each change takes time. Chapter 6 tells how to compile the files that do not change once, and keep them in a library.

.COM and .EXE Programs

A .COM program is made of one segment of 64 KB of the computer's memory, shared by its code, its variables, its heap and its stack. For most programs that is plenty. For one that needs more data, -e makes an .EXE: its code is in a segment, and its data, variables, heap and stack have a segment of 64 KB all to themselves. This program has 60000 bytes of data:

```
/* BIG.C - more data than a .COM program can hold: make it with SMALLC -e */
#include <stdio.h>

char big[30000];
int more[15000];

int main(void)
{
    int i;
    for (i = 0; i < 30000; i++)
        big[i] = i;
    for (i = 0; i < 15000; i++)
        more[i] = i * 3;
    printf("%u bytes of data, big[29999] = %d, more[14999] = %d\n",
           sizeof(big) + sizeof(more), big[29999], more[14999]);
    return 0;
}
```

As a .COM program it is too big to start: the stack needs room too. As an .EXE, it works. COS looks for a .COM with the name you type before it looks for an .EXE, so the old `BIG.COM` is deleted first:

```
A:\>smallc big.c
BIG.COM: 4100 bytes

A:\>big
Not enough memory

A:\>smallc -e big.c
BIG.EXE: 4180 bytes

A:\>del big.com

A:\>big
60000 bytes of data, big[29999] = 47, more[14999] = -20539

A:\>
```

An array has from 1 to 32767 elements; keep each array and structure under 32 KB, in a .COM or an .EXE. The code of a program may not be larger than 64 KB, in either. Appendix B describes how both kinds of program use the memory.

Where Small-C Finds Its Files

Small-C is told where it is when COS runs it, and looks for what it needs there:

- **`#include "name"`** looks first in the directory of the file that has the `#include`, then in the directories given with -I, in order, then in the directory INCLUDE of the directory where Small-C is, then in the directory where Small-C is. **`#include <name>`** does the same, but skips the first step. On the System Diskette the header files are in the root directory, next to `SMALLC.EXE`.
- **`CLIB.LIB`**, the library, is in the directory where Small-C is.
- **`ASM.COM`** is in the directory where Small-C is; if it is not there, in the root directory of the current drive.

So Small-C works from any directory, if you type its whole name: A:\SMALLC. Small-C writes its files, `CCTEMP.ASM` and the program, in the current directory.

How Small-C Makes a Program

Small-C does not make the .COM file by itself. It writes the whole program as one source in assembly language, in the file `CCTEMP.ASM`, and then runs ASM, the COS assembler, which makes the program from it and shows its name and size. Then Small-C deletes `CCTEMP.ASM`. The source holds, one after the other:

1. the start-up module, which sets the program going and calls *`main`* (Appendix B);
2. the files that you gave, in order: the C sources compiled, the assembly files copied;
3. the modules of the libraries that the program needs, and those that they need;
4. the end of the program, where the variables that start as zero are placed.

The computer works at 4.77 megahertz, and Small-C and ASM have much to do: SMALLC `HELLO.C` takes about 50 seconds, of which about 20 are Small-C reading and compiling and 30 are ASM assembling. A program that prints something, like this one, brings in the library functions that print, and that is most of the time. The compiled program starts at once, and is fast.

The programs that ASM finds mistakes in are those that have assembly language of their own. ASM shows its own messages (Chapter 10 of the COS User's Guide), with the line in `CCTEMP.ASM`. If you wish to read what Small-C wrote, use -S: it keeps the .ASM file, with the name of the program, and does not run ASM. You can then assemble it yourself with ASM.

```
A:\>smallc -S -o sq.com squares.c

A:\>dir sq.*
```

```
 Volume in drive A is COS
 Directory of  A:\

SQ       ASM    32836   1-01-80  12:00a
        1 File(s)   1211904 bytes free

A:\>
```


The Command Line of a Program

The words typed after the name of a program arrive in *main* as `argc` and `argv`, as the language says: `argv`[0] is the name of the program with its whole path, `argv`[1] is the first word, and so on. A word in quotation marks may have spaces inside; the marks are not part of the word. A program may have 24 words, and the whole line may have 127 characters.

```
/* ARGS.C - shows the words of the command line */
#include <stdio.h>

int main(int argc, char **argv)
{
    int i;

    for (i = 0; i < argc; i++)
        printf("argv[%d] = \"%s\"\n", i, argv[i]);
    return argc - 1;
}
```

```
A:\>smallc args.c
ARGS.COM: 4024 bytes

A:\>args one "two words" 3
argv[0] = "A:\ARGS.COM"
argv[1] = "one"
argv[2] = "two words"
argv[3] = "3"

A:\>
```

The value that *main* returns, or that exit is given, is the return code of the program. A program that runs another one with `spawn` `gets` it (Chapter 5).

This page intentionally left blank.

3

The Language

Contents

What Small-C Accepts 3-2
Types and Constants 3-2
Declarations 3-4
Expressions 3-4
Arrays and Pointers 3-5
Structures, Unions and Enumerations 3-6
Functions 3-7
Statements 3-9
What Small-C Does Not Have 3-10

What Small-C Accepts

Part	What there is
Types	char, unsigned char, int, short, unsigned, pointers, arrays of any number of dimensions, struct, union, enum, void.
Storage	static, extern, auto and register. const and volatile are accepted and ignored.
Functions	Prototypes and old-style definitions, variable arguments (...), recursion. A function called before it is declared returns int.
Operators	All those of C for integers and pointers: arithmetic, bits, comparisons, && || !, ?:, comma, assignments, ++ and --, casts, sizeof, &, *, [], . and ->.
Statements	if, else, while, do, for, switch, case, default, break, continue, goto, return, labels, blocks.
Constants	Decimal, octal (017), hexadecimal (0x1F), character constants, strings.
Preprocessor	#include, #define, #undef, #if, #elif, #else, #endif, #ifdef, #ifndef, #error, #pragma, and #asm (Chapter 4).

Types and Constants

Type	Size	Values
char	1 byte	-128 to 127. The sign counts: char is signed.
unsigned char	1 byte	0 to 255.
int, short	2 bytes	-32768 to 32767. short is the same type as int.
unsigned, unsigned int	2 bytes	0 to 65535.
pointer	2 bytes	An address in the data segment of the program. Pointers compare as unsigned numbers.
enum	2 bytes	An int. The names of an enumeration are constants.

In an expression a char is turned into an int first, with its sign. When an operation has an unsigned operand the operation is unsigned: division, remainder, shifts to the right and comparisons. The results of integer arithmetic that do not fit in 16 bits are cut to 16 bits, without any message.

A decimal constant above 32767, or one that ends in U, is `unsigned`. A constant with L is an error: there are no `long` numbers. A character constant like 'A' has the code of the character; the escapes are \n \t \r \a \b \f \v \0, \x followed by hexadecimal digits and \ followed by up to three octal digits. Strings written next to each other are joined into one: "ab" "cd" is "abcd". A string may have up to 299 characters, and each one written in a program is kept separately, in the data of the program, where it can be changed.


```
/* TYPES.C - the integer types and how they mix */
#include <stdio.h>

int main(void)
{
    char c;
    unsigned char b;
    int i;
    unsigned u;

    c = 200;                    /* char is signed: this is -56 */
    b = 200;
    i = -7;
    u = 65535;
    printf("%d %d %d\n", c, b, (unsigned char)c);
    printf("%d %d %d\n", i / 2, i % 2, i >> 1);
    printf("%u %u %d\n", u >> 1, u + 1, (int)u);
    printf("%d %d\n", i < 1, (unsigned)i < 1);
    printf("%d %d\n", u > 30000, sizeof(int[10]));
    printf("%d %d %d\n", 0x1F, 017, 'A');
    return 0;
}
```

```
A:\>smallc types.c
TYPES.COM: 4198 bytes

A:\>types
-56 200 200
-3 -1 -4
32767 0 -1
1 0
1 20
31 15 65

A:\>
```

The first line shows the sign of `char` (200 is -56 when it is a `char`), the second that division truncates toward zero and the shift of a negative number keeps the sign, and the third the `unsigned` numbers.

Declarations

A declaration is as in C: **`int a, *b, c[10];`** declares an `int`, a pointer to `int` and an array of ten. Inside a function the declarations must be at the start of a block, before the first statement; Small-C says "declarations go at the start of a block" otherwise. A variable declared in a block is known until the end of the block.

Initial values

- Variables outside functions, and **`static`** variables of functions, start as zero, or with the value given. The value must be constant: numbers and constant expressions, the address of a variable (&x, &table[2], or the name of an array), or a string.
- Arrays and structures are initialized with braces, as in C, with braces inside braces, with fewer values than elements (the others are zero) and with a last comma. If the size of an array is left out, the values give it. A `char` array may be initialized with a string, and a `char` pointer too.
- Variables of a function (`auto`) may be initialized with any expression, when they are simple numbers or pointers. An array or structure of a function cannot have an initial value: make it `static`.
- One initializer may have up to 40 strings for pointers, with 600 characters in all. A larger table of strings is better in two parts.

Declarators

Pointers and arrays can be combined as in C (an array of pointers is **`char *names[4]`**), but parentheses in a declarator are not accepted, so there are no pointers to arrays, nor pointers to functions. An array has at most 8 dimensions. Only the first one may be left out.

Expressions

The operators are those of C, with the same precedence, in this order, from the tightest:

Operators	Meaning
`( ) [ ] . -> ++ --`	Call, index, members, postfix increment and decrement.
`! ~ - + * & ++ -- sizeof (type)`	Unary operators and casts.
`* / %`	Multiplication, division, remainder.
`+ -`	Addition, subtraction.
`<< >>`	Shifts.
`< <= > >=`	Relations.

Operators	Meaning
== !=	Equality.
& ^ |	Bitwise and, exclusive or, or (three levels, in this order).
&& ||	Logical and, or: the right side is worked out only if needed.
?:	Condition.
= += -= *= /= %= <<= >>= &= ^= |=	Assignments.
,	Comma.

The arguments of a function are worked out from the right to the left, and the order of the other parts of an expression is not defined: do not write i++ + ++i. A pointer plus or minus an integer moves by the size of what it points to; two pointers to the same kind of thing subtract to a count of elements; &, * and [] are as in C. Constant expressions are worked out when the program is compiled, and a division by zero in one is an error.

An expression may have up to 250 parts; a longer one gives "expression too complex" and has to be written in pieces.

Arrays and Pointers

The name of an array, used as a value, is a pointer to its first element. Arrays of arrays are stored row by row.

```
/* ARRAYS.C - arrays, pointers and strings */
#include <stdio.h>
#include <string.h>

int table[3][4];
char name[] = "Ada";
char *days[] = { "Mon", "Tue", "Wed" };

int main(void)
{
    int i, j, *p;
    char *s;

    for (i = 0; i < 3; i++)
        for (j = 0; j < 4; j++)
            table[i][j] = i * 10 + j;
    p = &table[1][0];
    printf("%d %d %d\n", table[2][3], p[2], *(p + 5));
    printf("%d %d\n", sizeof(table), sizeof table[0]);
    s = name;
    while (*s)
        printf("%c.", *s++);
```

```
    printf(" %d %s\n", strlen(name), days[2]);
    printf("%d\n", &table[2][0] - &table[0][0]);
    return 0;
}
```

```
A:\>smallc arrays.c
ARRAYS.COM: 5158 bytes

A:\>arrays
23 12 21
24 8
A.d.a. 3 Wed
8

A:\>
```


Structures, Unions and Enumerations

A structure is the members one after another, byte by byte, without any space between them; a `union` takes the size of its largest member. A structure may be assigned to another of the same type, and used through pointers (->), but it cannot be passed to a function or returned from one: pass its address. The tag of a structure may be used inside it, as the type of a pointer, for lists and trees. The names of an enumeration are `int` constants, counted from 0 or from the value given.

```
/* STRUCTS.C - structures, unions and enumerations */
#include <stdio.h>

enum color { RED, GREEN = 5, BLUE };

struct point {
    int x, y;
};

struct item {
    char name[8];
    struct point where;
    enum color shade;
};

union word {
    unsigned w;
    struct { unsigned char lo, hi; } b;
};

struct item items[2] = {
    { "key", { 3, 4 }, GREEN },
    { "lamp", { 10, 20 }, BLUE }
};
```

```
void move(struct item *it, int dx, int dy)
{
    it->where.x += dx;
    it->where.y += dy;
}

int main(void)
{
    struct item copy;
    union word w;

    copy = items[1];
    move(&copy, -1, 5);
    printf("%s %d,%d %d\n", copy.name, copy.where.x, copy.where.y, copy.shade);
    printf("%s %d,%d\n", items[1].name, items[1].where.x, items[1].where.y);
    w.w = 0x1234;
    printf("%x %x %d %d\n", w.b.lo, w.b.hi, sizeof(struct item), sizeof(union w
    return 0;
}
```

```
A:\>smallc structs.c
STRUCTS.COM: 4164 bytes

A:\>structs
lamp 9,25 6
lamp 10,20
34 12 14 2

A:\>
```

The size of `struct` item is 14: 8 for the name, 4 for the point and 2 for the enumeration.

Functions

A function may be written in the new way, with the types in the parentheses, or the old way, with the names in the parentheses and the types after them. A prototype may leave out the names of the parameters; a definition may not. A function with no value is **`void`**, and one with no parameters has **`(void)`**, or empty parentheses. A function may have up to 32 parameters.

Small-C does not compare the arguments of a call with the prototype: neither their number nor their types. A call with the wrong number of arguments compiles, and does nonsense when it runs; be careful. The prototype is used for the type of the value that the function returns. A function that is called before it is declared is taken for one that returns `int`, so declare functions that return anything else before they are used.

Variable arguments are written with ... and read with the macros of `STDARG.H`. The arguments are words, so a `char` is read as an `int`.

```
/* FUNCS.C - functions: prototypes, old style, variable arguments */
#include <stdio.h>
#include <stdarg.h>

int sum(int count, ...)
{
    va_list ap;
    int total;

    va_start(ap, count);
    total = 0;
    while (count-- > 0)
        total += va_arg(ap, int);
    va_end(ap);
    return total;
}

int gcd(a, b)
int a, b;
{
    return b ? gcd(b, a % b) : a;
}

int next(void)
{
    static int n = 100;

    return n++;
}

int main(void)
{
    int r;

    r = half(9);                /* not declared: it returns int */
    printf("%d %d %d\n", sum(4, 10, 20, 30, 40), gcd(84, 36), r);
    printf("%d %d\n", next(), next());
    return 0;
}

half(x)
{
    return x / 2;
}
```

```
A:\>smallc funcs.c
FUNCS.COM: 4174 bytes

A:\>funcs
100 12 4
101 100
```

```
A:\>
```

The `static` variable of next keeps its value between calls, and starts at 100. The function half was not declared when `main` called it; it has no type, so it returns `int`.

Statements

All the statements of C are there. A switch takes an integer; the values of the cases are constant expressions, the same value twice is an error, and a function may have 150 cases in all. A break leaves the innermost loop or switch, a continue goes to the next turn of the innermost loop (a switch inside a loop does not stop it). A label is a name followed by a colon, `goto` goes to a label of the same function, and names of labels are apart from all the other names. Loops and switches may be nested 32 deep.


```
/* FLOW.C - switch, goto and the other statements */
#include <stdio.h>

int vowels(char *s)
{
    int n;

    n = 0;
    for (; *s; s++)
        switch (*s) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
            n++;
            break;
        default:
            break;
        }
    return n;
}

int main(void)
{
    int i, j;

    printf("%d\n", vowels("education"));
    for (i = 0; i < 4; i++)
        for (j = 0; j < 4; j++)
            if (i * j == 6)
                goto found;
    printf("none\n");
    return 1;
found:
    printf("%d x %d\n", i, j);
    i = 0;
    do {
```

```
        i += 3;
    } while (i < 10);
    printf("%d %d\n", i, i > 10 ? 1 : 2);
    return 0;
}
```

```
A:\>smallc flow.c
FLOW.COM: 4208 bytes

A:\>flow
5
2 x 3
12 1

A:\>
```


What Small-C Does Not Have

Each of these gives an error message when a program uses it:

Not there	Message	What to do
`long`	`long` is not supported; `long` constants are not supported	Use `int` or `unsigned`, or keep a number in two words.
`float, double`	floating point is not supported	Use whole numbers: hundredths instead of fractions, for example.
`typedef`	`typedef` is not supported	Use `#define` for simple names, and write `struct` tags.
`Pointers to functions`	pointers to functions are not supported	Use a switch or a table of numbers. For this reason the library has no `qsort`.
`Bit fields`	bit fields are not supported	Use shifts and masks.
`Structures by value`	structures are passed by pointer, not by value; functions return structures by pointer, not by value	Pass and return their addresses.
`Initial values of arrays and structures in a function`	only simple locals can be initialized here; make it `static`	Declare it `static`.
`Declarations after statements`	declarations go at the start of a block	Start a block with { where the variable is needed.

Not there	Message	What to do
`The # and ## operators of macros`	the # and ## operators are not supported	Write the macro in another way.

Also missing, but without a message of their own: the library has no `scanf`, no `fseek` (see `lseek` in `IO.H`), no floating point functions, no time functions.

This page intentionally left blank.

4

The Preprocessor

Contents

How the Preprocessor Works 4-2
Macros 4-2
Conditions 4-3
Predefined Names 4-3

How the Preprocessor Works

Small-C reads a program a line at a time. Comments are taken out (both /* ... */, which does not nest, and // to the end of the line), a line that begins with # is a directive and is done at once, and in all the other lines macros are replaced by what they stand for. A line ending in a backslash goes on in the next. A line may have up to 511 characters, and 767 after its macros are replaced.

Directive	Does
`#include "file"` `#include <file>`	Reads another file here. Where Small-C looks is told in Chapter 2. Files may include files up to 5 levels deep.
`#define name text` `#define name(a, b) text`	Defines a macro. Defining a name again replaces it.
`#undef name`	Forgets a macro.

`#if expression`
`#elif expression`
`#else`
`#endif`
	Keeps or skips lines. Up to 16 levels.

`#ifdef name` `#ifndef name`	`#if` for a name that is, or is not, a macro.
`#error text`	Makes Small-C show the text as an error message.
`#pragma, #line`	Accepted and ignored.
`#asm ... #endasm`	Lines of assembly language that go into the program (Chapter 5).

Macros

A macro without parameters is replaced by its text wherever its name is found as a word, outside of strings. A macro with parameters is replaced only when its name is followed by an opening parenthesis, and the arguments, separated by commas (commas in parentheses do not count), must be on one line. A macro may use other macros, but one that uses itself is stopped by an error ("a macro expands without end"). A macro may have up to 16 parameters. In a source file, the macros may have 3000 characters of text in all, and there may be 250.

Put parentheses around the parameters and around the whole text of a macro that is an expression: SQUARE(n + 1) must be ((n + 1) * (n + 1)), not n + 1 * n + 1.

Conditions

The expression of `#if` is made of whole numbers and character constants, the operators of C except the assignments and the comma, and parentheses. **defined(name)** (or defined name) is 1 for a macro and 0 for any other name; a name that is not a macro counts as 0. Its numbers are 16-bit as everywhere in Small-C.

Predefined Names

`__COS__`	1. The program is made for COS.
`__SMALLC__`	1. The program is made by Small-C.

Macros can also be defined on the command line: **smallc -DSIZE=10 -DDEBUG pre.c** is the same as `#define` SIZE 10 and `#define` DEBUG 1 before the first line of each file.

```
/* DEFS.H - shared definitions */
#define SQUARE(x)   ((x) * (x))
#define MAX(a, b)   ((a) > (b) ? (a) : (b))
#define VERSION     2
```

```
/* PRE.C - the preprocessor */
#include <stdio.h>
#include "defs.h"

#ifndef SIZE
#define SIZE 4
#endif

#if VERSION >= 2 && defined(DEBUG)
#define SHOW(x) printf("debug\n")
#elif VERSION >= 2
#define SHOW(x) printf("v2: %d\n", x)
#else
#define SHOW(x)
#endif

int buffer[SIZE];

int main(void)
{
    int n;

    n = 3;
    SHOW(SQUARE(n + 1));
    printf("%d %d\n", MAX(n, 10), sizeof(buffer) / sizeof(buffer[0]));
#ifdef __COS__
    printf("running under COS, compiled by Small-C %d\n", __SMALLC__);
#endif
```

```
    return 0;
}
```

```
A:\>smallc pre.c
PRE.COM: 4086 bytes

A:\>pre
v2: 16
10 4
running under COS, compiled by Small-C 1

A:\>smallc -DSIZE=10 -DDEBUG pre.c
PRE.COM: 4068 bytes

A:\>pre
debug
10 10
running under COS, compiled by Small-C 1

A:\>
```

With the second command the buffer has 10 elements, and the debug version of SHOW is the one chosen.

5

Using the Machine

Contents

What a C Program Can Reach 5-2
Ports 5-2
Memory 5-3
Interrupts 5-4
The Keyboard and the Screen 5-5
Assembly Language in C 5-5
Running Other Programs 5-6
Memory for Data 5-7

What a C Program Can Reach

The functions of DOS.H and CONIO.H give a C program the same reach that an assembly language program has. They are described in Chapter 7; this chapter shows how they are used. The computer's ports, memory, BIOS and COS services are described in the COS User's Guide (Chapters 6 to 8 and Appendix D).

Ports

inp and outp read and write a byte at a port of the computer; inpw and outpw, a word. This program makes a sound with the timer and the speaker, as Appendix D of the COS User's Guide describes them, and waits for 9 ticks of the clock before stopping it.


```
/* TICKS.C - ports, memory and interrupts: a beep and a clock */
#include <stdio.h>
#include <dos.h>
#include <conio.h>

unsigned ticks(void)
{
    return peek(0, 0x46C);          /* the BIOS counts 18.2 ticks a second */
}

void wait(int n)
{
    unsigned t;

    t = ticks();
    while (ticks() - t < n)
        ;
}

void tone(int divisor)
{
    outp(0x43, 0xB6);               /* timer 2, a square wave */
    outp(0x42, divisor & 255);
    outp(0x42, divisor >> 8);
    outp(0x61, inp(0x61) | 3);      /* the speaker on */
}

void quiet(void)
{
    outp(0x61, inp(0x61) & 0xFC);
}

int main(void)
{
    struct REGS r;
    int n;
```

```
    r.ax = 0x3000;                  /* COS function 30h: the version */
    intdos(&r, &r);
    printf("COS version %d.%d\n", r.ax & 255, r.ax >> 8);
    tone(2712);                     /* about 440 Hz */
    wait(9);
    quiet();
    printf("Counting seconds; press a key to stop.\n");
    for (n = 1; n <= 5 && !kbhit(); n++) {
        wait(18);
        printf("%d second%s\n", n, n == 1 ? "" : "s");
    }
    if (kbhit())
        getch();
    return 0;
}
```

```
A:\>smallc ticks.c
TICKS.COM: 4462 bytes

A:\>ticks
COS version 3.30
Counting seconds; press a key to stop.
1 second
2 seconds
3 seconds
4 seconds
5 seconds

A:\>
```


Memory

peek and `peekb` read a word and a byte at any segment and offset; poke and `pokeb` write them. They make the BIOS data (segment 0x40, or segment 0 and offsets from 0x400: the tick counter above is at 0:0x46C) and the video memory available:

```
/* VIDEO.C - the screen: video memory and the BIOS */
#include <stdio.h>
#include <dos.h>

/* Puts a character and its color at a row and column (80 x 25 text) */
void put(int row, int col, int ch, int attr)
{
    unsigned off;

    off = (row * 80 + col) * 2;
    pokeb(0xB800, off, ch);
    pokeb(0xB800, off + 1, attr);
}
```

```
int main(void)
{
    struct REGS r;
    char *s;
    int i;

    s = "Written straight into video memory";
    for (i = 0; s[i]; i++)
        put(10, 20 + i, s[i], 0x1E);        /* yellow on blue */
    r.ax = 0x0200;                          /* BIOS 10h: set the cursor */
    r.bx = 0;                               /* page 0 */
    r.dx = 0x0C00;                          /* row 12, column 0 */
    int86(0x10, &r, &r);
    printf("The cursor was moved with INT 10h.\n");
    return 0;
}
```

```
A:\>smallc video.c
VIDEO.COM: 4278 bytes

A:\>video




                    Written straight into video memory

The cursor was moved with INT 10h.

A:\>
```


The characters of the text screen are at segment 0xB800, a character and then its color byte (yellow on blue is 0x1E). Pointers of C reach only the data segment of the program; `getds` gives its segment, if you need to say it to a service, and the variable `_psp`, declared in `DOS.H`, the segment of the program's PSP.

Interrupts

`int86`(*n*, *&in*, *&out*) loads the registers AX, BX, CX, DX, SI and DI from the structure *in*, runs INT *n*, and `puts` the registers that the service gave back into *out*; the structure has the carry flag too, in cflag (1 when the service failed, with an error code in ax). `intdos` does the same for INT 21h, the COS services. The two structures may be the same one, as in the examples. DS is the data segment of the program, so a pointer that is given in DX, SI or DI is understood by the service; the other segment registers are not set.

```
struct REGS r;

r.ax = 0x3000;              /* function 30h: the version of COS */
intdos(&r, &r);
printf("%d.%d\n", r.ax & 255, r.ax >> 8);
```

The number of the interrupt is written into the code of `int86`, so it may be any variable.

The Keyboard and the Screen

`printf` and the other functions of `STDIO.H` write on the screen and read the keyboard line by line, as COS does it. For a program that wants a key at once, without waiting for Enter, `CONIO.H` has `getch`, that waits for a key and shows nothing, `getche`, that shows it, and `kbhit`, that tells whether a key is waiting, so that a program can do something else until it is. Keys without a character, like the function and cursor keys, come as two values: 0, and then a code (Appendix B of the COS User's Guide). `getch` does not stop the program when you press Ctrl-C: it is the character 3, and the program must end by itself. `putch` and `cputs` write characters just as they are; end a line with both \r and \n there.

`stdout` is written at the end of each line, and before the program waits for the keyboard. A program that writes a part of a line, and then waits with `getch`, should call `fflush(stdout)` first.

Assembly Language in C

Lines between `#asm` and `#endasm` go into the program as they are. The lines are not looked at by the preprocessor; they are limited to 767 characters in all. There are two ways to use them.

Inside a function, the lines become part of its code. BP holds the frame of the function: the first parameter is at [bp+4], the second at [bp+6], and so on, and what the function returns is in AX. In the first function below there is no return statement; the value that is in AX when the function ends is the result.

At the top level of a file, the lines are a function (or data) of their own. Write **section .text** first, give the label with an underscore, as in _swapb, and tell Small-C its name with a line **`;#def _swapb`**, so that the program can find it (Chapter 6). Here the arguments are at [sp+2], [sp+4]...: BX is the easy way to read them. A function must leave BP, SI, DI, DS and SS as it found them (Appendix B).

```
/* ASMDEMO.C - assembler inside a C program */
#include <stdio.h>

/* A whole function in assembler, at the top level of the file.
   The arguments are at [sp+2], [sp+4]...; the result goes in AX. */
```

```
#asm
section .text
;#def _swapb
_swapb:
    mov bx, sp
    mov ax, [bx+2]
    xchg al, ah
    ret
#endasm

unsigned swapb(unsigned x);

/* A few instructions inside a C function. BP holds the frame, so the
   first argument is at [bp+4]; the value of the function is AX. */
int triple(int x)
{
#asm
    mov ax, [bp+4]
    mov cx, ax
    add ax, ax
    add ax, cx
#endasm
}

int main(void)
{
    printf("%x %d\n", swapb(0x1234), triple(14));
    return 0;
}
```



```
A:\>smallc asmdemo.c
ASMDEMO.COM: 4006 bytes

A:\>asmdemo
3412 42

A:\>
```

Running Other Programs

spawn(*path*, *args*) runs a program, waits for it to end and returns its return code, or -1 if it could not be run. The name needs its extension, .COM or .EXE, and COS does not look for it in other directories. *args* is the command line that the program will see after its own name. A .COM program made by Small-C keeps only 64 KB and gives back all the other memory, so the program that it starts has room.

```
/* RUN.C - runs another program and reports its return code */
#include <stdio.h>
#include <dos.h>
```

```
int main(void)
{
    int code;

    code = spawn("ARGS.COM", "a b");
    printf("ARGS ended with %d\n", code);
    code = spawn("NOSUCH.COM", "");
    printf("NOSUCH gives %d\n", code);
    return 0;
}
```

```
A:\>smallc args.c
ARGS.COM: 4024 bytes

A:\>smallc run.c
RUN.COM: 4308 bytes

A:\>run
argv[0] = "A:\ARGS.COM"
argv[1] = "a"
argv[2] = "b"
ARGS ended with 2
NOSUCH gives -1

A:\>
```


The return code of ARGS is 2: its `argc` less one. Return codes are 0 to 255.

Memory for Data

`malloc` takes memory from the heap, which goes from the end of the variables of the program to 2 KB below the stack, and free gives it back. The stack grows down from the top of the segment toward the heap, so a program with deep recursion and a big heap may find them touching: keep the 2 KB free. A program that has not got 2 KB of room between its variables and its stack ends at the start with "Not enough memory". The variables that are not initialized start as zero; the memory that `malloc` gives does not (use `calloc` for that).

This page intentionally left blank.

6

Modules and Libraries

Contents

Modules 6-2
Making a Library 6-2
Using a Library 6-2
Rules 6-3
Modules in Assembly Language 6-3
Start-up Modules 6-4

Modules

A *module* is a piece of a program in assembly language, with a few lines that tell the link what the module has and needs. Small-C writes a module for each C source it compiles. The lines begin with ;# (they are comments for ASM) and say:

Line	Meaning
`;#module` *NAME*	A module begins here. Small-C names a module with the name of its file, in capitals, up to 8 letters and digits.
`;#def` *names*	The names that the module defines. They come before the `;#ref` lines.
`;#ref` *names*	The names that the module uses and does not define.
`;#bss` *name size*	A variable of that many bytes, that starts as zero. The link places all of them at the end of the program.
`;#weak` *name*	A function that may be missing: if nobody defines it, it is a function that does nothing (a single RET). The start-up uses this for the stream functions.

The names of C functions and variables have a leading underscore: `main` is _main. Appendix B shows a whole module, as Small-C writes it:

Making a Library

A library is the modules, one after the other, in one file. -c with -o makes it from C files and assembly language files:

Format

```
SMALLC -c -o library.LIB file ...
```

Without -o, -c makes a .ASM file for each .C file, as in **smallc -c vec.c**, which writes `VEC.ASM`, a module of its own. The .ASM files given with -o must have their own `;#module` line, so that the library knows where each begins.

Using a Library

A library is given to Small-C as any other file, or with -L. Small-C does not take all of it: it takes the modules that define names that the program uses, and the modules that those need, and no others. A program is so as small as it can be, which is also why `CLIB.LIB` can hold every function of Chapter 7 without making every program larger.

```
A:\>smallc -c vec.c

A:\>smallc -c -o vec.lib vec.c
```

```
A:\>smallc demo.c vec.lib
DEMO.COM: 4190 bytes

A:\>demo
sum 583, largest 358
2 calls

A:\>dir vec.*

 Volume in drive A is COS
 Directory of  A:\

VEC      C        371   1-01-80  12:00a
VEC      H        122   1-01-80  12:00a
VEC      ASM     1253   1-01-80  12:00a
VEC      LIB     1253   1-01-80  12:00a
        4 File(s)   1236480 bytes free

A:\>
```

VEC.ASM and VEC.LIB are the same here, because the library has one module. Compiling VEC.C once and keeping it in a library saves Small-C the work of reading it each time; ASM still has to assemble the whole program.

Rules

- A name must be defined once. Two modules that define the same function, or give the same variable a value, give the message "*name* is defined in two modules". A module taken from a library is chosen only if no module that you gave defines the name.
- A name that nobody defines gives "*name* is not defined anywhere", with the name as written in C.
- A library holds up to 80 modules and 600 names in all; a program may have 250 variables made by ;#bss.
- Static functions and variables are named MODULE@name (Appendix B), so two modules may each have their own.

Modules in Assembly Language

A function written in assembly language is a module like the others. It needs the ;#def line for what it defines and a ;#ref line for each library function that it calls, or the link will not bring that function in. The addresses of data are written as *label* - __dbase, which is right in .COM and .EXE programs (Appendix B), and arguments are removed from the stack by the caller.

```
;#module GREET
; GREET.ASM - greet(): prints a line with puts
;#def _greet
;#ref _puts
section .text
_greet:
    mov ax, message - __dbase       ; the address of the text, in the data segme
    push ax
    call _puts                      ; puts(message)
    add sp, 2                       ; the caller takes the arguments off
    ret
section .data
message db 'Greetings from assembler!', 0
```

```
/* USEGREET.C - calls a function written in assembler */
void greet(void);

int main(void)
{
    greet();
    return 0;
}
```



```
A:\>smallc usegreet.c greet.asm
USEGREET.COM: 2650 bytes

A:\>usegreet
Greetings from assembler!

A:\>
```

The same module can be given in a library: **smallc -c -o g.lib greet.asm**, and then **smallc usegreet.c g.lib**.

Start-up Modules

Every program begins with a start-up module that `CLIB.LIB` has in two forms: C0COM for .COM programs and C0EXE for .EXE programs. It sets up the segments, clears the variables, builds `argc` and `argv`, calls *`main`* and ends the program with the value it returned. -n leaves `CLIB.LIB` out; the libraries given with -L must then have a start-up, and everything the program calls.

7

Library Reference

Contents

How to Read This Chapter 7-2
STDIO.H 7-2
STDLIB.H 7-4
STRING.H 7-5
CTYPE.H 7-6
STDARG.H 7-6
IO.H 7-7
DOS.H 7-8
CONIO.H 7-8

How to Read This Chapter

The functions are grouped by the header file that declares them, and are in alphabetic order in each group. Include the header before using its functions. Functions that fail return 0 (for pointers), EOF or -1, as each one says.

STDIO.H: Streams

A *stream* is a file, or the keyboard and screen, with a buffer of its own. stdin, stdout and stderr are open when the program starts; at most 7 more files may be open at once (FOPEN_MAX is 10, counting those three). A stream opened for text turns each \n written into the two characters CR LF, and drops the CR of CR LF when reading; a Ctrl-Z in the file is the end of the text. A stream opened with "b" does neither. stdout and stderr are written at each \n and before stdin is read; every stream is written when the program ends with exit, or by return from main. A program that ends with _exit loses what is still in its buffers.

STDIO.H defines FILE, EOF (-1), BUFSIZ (512), FOPEN_MAX (10) and NULL (0).

Function	Does
`void clearerr(FILE *f)`	Clears the end-of-file and error marks of the stream.
`int fclose(FILE *f)`	Writes what is waiting in the buffer, closes the file and frees the buffer. 0, or EOF if something went wrong.
`int feof(FILE *f)`	Not 0 if the end of the file was reached by a read.
`int ferror(FILE *f)`	Not 0 if a read or a write failed.
`int fflush(FILE *f)`	Writes what is waiting in the buffer of an output stream. 0, or EOF on an error.
`int fgetc(FILE *f)`	The same as getc.
`char *fgets(char *s, int n, FILE *f)`	Reads a line, up to n-1 characters, with its \n if it fits, and ends it with a zero. Gives s, or 0 at the end of the file when nothing was read.
`FILE *fopen(char *name, char *mode)`	Opens a file. mode begins with **r** (read; the file must exist), **w** (write; the file is made, or emptied) or **a** (write at the end; the file is made if it is not there); then may have **+** (read and write) and **b** (not text). Gives 0 if it cannot: no file, no free stream, no memory for the buffer.
`int fprintf(FILE *f, char *format, ...)`	printf to a stream.
`int fputc(int c, FILE *f)`	The same as putc.

Function	Does
`int fputs(char *s, FILE *f)`	Writes a string, without a \n. 0, or `EOF`.
`int fread(void *buf, int size, int n, FILE *f)`	Reads n items of size bytes. Gives the number of whole items read.
`int fwrite(void *buf, int size, int n, FILE *f)`	Writes n items of size bytes. Gives the number written.
`int getc(FILE *f)`	The next character of the stream, 0 to 255, or `EOF` at the end. `getc` and `putc` are functions, not macros.
`int getchar(void)`	`getc(stdin)`.
`char *gets(char *s)`	Reads a line from `stdin`, without its \n. It does not know how `long` s is: use `fgets` when the line may be `long`.
`int printf(char *format, ...)`	Writes formatted text on `stdout`. Gives the number of characters written. See below.
`int putc(int c, FILE *f)`	Writes a character. Gives it, or `EOF`.
`int putchar(int c)`	`putc(c, stdout)`.
`int puts(char *s)`	Writes a string and a \n on `stdout`.
`int remove(char *name)`	Deletes a file. 0, or -1.
`int rename(char *old, char *new)`	Renames a file. 0, or -1.
`int sprintf(char *s, char *format, ...)`	`printf` into a string, ended with a zero. The string must be large enough.
`int ungetc(int c, FILE *f)`	Gives one character back to be read again. Only one may be waiting.

`int vfprintf(FILE *f, char *format, char *ap)`
`int vprintf(char *format, char *ap)`
`int vsprintf(char *s, char *format, char *ap)`
	As `fprintf`, `printf` and `sprintf`, with the arguments read from ap, a `va_list` that `va_start` made (`STDARG.H`).

The formats of `printf`

A conversion is % followed by, in this order: flags, a width, a point and a precision, and a letter.

`%d %i`	An `int` in decimal, with its sign.
`%u`	An `unsigned` number in decimal.
`%x %X`	An `unsigned` number in hexadecimal, with small or capital letters.

%o	In octal.
%c	A character.
%s	A string. A null pointer shows as (null). A precision is the most characters to show.
%p	A pointer, in hexadecimal, at least four digits.
%%	A % sign.

The flags: **-** puts the value at the left of its width, **0** fills a number with zeros, **+** shows the sign of positive numbers and a space shows a space in its place. The width is a number or *, which takes it from the next argument; so may the precision, after the point. With a number, the precision is the least digits (up to 8). The letters l and h are accepted and ignored: numbers are always 16 bits. There is no conversion for fractions.

```
printf("[%5d][%-5d][%05d][%x][%c][%.2s]\n", 42, 42, 42, 255, 'Z', "abc");
[   42][42   ][00042][ff][Z][ab]
```

STDLIB.H: Memory and Numbers

STDLIB.H defines NULL, RAND_MAX (32767), EXIT_SUCCESS (0) and EXIT_FAILURE (1).

Function	Does
`int abs(int v)`	The absolute value.
`int atoi(char *s)`	The number at the start of s: spaces, a sign and digits. Stops at the first other character.
`void *calloc(unsigned n, unsigned size)`	Memory for n items of size bytes, set to zero. 0 if there is not enough.
`void exit(int code)`	Writes all the streams and ends the program, with the return code.
`void _exit(int code)`	Ends the program at once, without writing the streams.
`void free(void *p)`	Gives memory back. free(0) does nothing.
`char *getenv(char *name)`	The value of a COS environment variable (see SET), or 0. The text is kept in a place that the next call changes.
`char *itoa(int v, char *s, int radix)`	Writes v in s in base radix (2 to 36, small letters). A sign only in base 10; in the other bases the number is unsigned. Gives s.
`void *malloc(unsigned n)`	Memory for n bytes, not cleared. 0 if there is not enough.
`int rand(void)`	The next number of a series of pseudo-random numbers, 0 to 32767.

Function	Does
`void *realloc(void *p, unsigned n)`	Changes the size of a block of memory, keeping what it held (the block may move). With p = 0, it is `malloc`. Gives 0, and leaves the block as it was, if there is no room.
`void srand(unsigned seed)`	Starts the series again at seed. At the start of a program, the seed is 1.

`STRING.H`: Strings and Memory

Function	Does
`int memcmp(void *a, void *b, int n)`	Compares n bytes: 0 if equal, else the difference of the first two that differ, as `unsigned` bytes.
`void *memcpy(void *d, void *s, int n)`	Copies n bytes. The areas may overlap. Gives d.
`void *memmove(void *d, void *s, int n)`	The same as `memcpy`.
`void *memset(void *d, int c, int n)`	Sets n bytes to c. Gives d.
`char *strcat(char *d, char *s)`	Adds s at the end of d. Gives d.
`char *strchr(char *s, int c)`	The first c in s, or 0. `strchr`(s, 0) is the end of the string.
`int strcmp(char *a, char *b)`	Compares strings: 0 if equal, negative if a is first, positive if b is.
`char *strcpy(char *d, char *s)`	Copies s, with its zero, to d. Gives d.
`char *strdup(char *s)`	A copy of s in memory from `malloc`, or 0. Free it when it is no longer wanted.
`int stricmp(char *a, char *b)`	As `strcmp`, with capitals and small letters the same.
`int strlen(char *s)`	The number of characters before the zero.
`char *strlwr(char *s)`	Turns s into small letters, in place. Gives s.
`char *strncat(char *d, char *s, int n)`	Adds at most n characters of s to d, and a zero. Gives d.
`int strncmp(char *a, char *b, int n)`	`strcmp` of at most n characters.

Function	Does
`char *strncpy(char *d, char *s, int n)`	Copies exactly n characters: if s is shorter, d is filled with zeros; if it is longer, d is not ended with a zero. Gives d.
`char *strrchr(char *s, int c)`	The last c in s, or 0.
`char *strstr(char *s, char *t)`	Where the string t is in s, or 0.
`char *strupr(char *s)`	Turns s into capitals, in place. Gives s.

CTYPE.H: Kinds of Characters

These functions take a character code and give not 0 if it is of the kind, and 0 if not. They know the ASCII characters only.

Function	True for
`isalnum(c)`	Letters and digits.
`isalpha(c)`	A to Z and a to z.
`iscntrl(c)`	Codes 0 to 31 and 127.
`isdigit(c)`	0 to 9.
`islower(c)`	a to z.
`isprint(c)`	Codes 32 to 126.
`ispunct(c)`	Printing characters that are not a space, a letter or a digit.
`isspace(c)`	Space, tab, line feed, vertical tab, form feed, carriage return.
`isupper(c)`	A to Z.
`isxdigit(c)`	0 to 9, A to F, a to f.
`tolower(c)`	The small letter of a capital; any other c unchanged.
`toupper(c)`	The capital of a small letter; any other c unchanged.

STDARG.H: Variable Arguments

For a function declared with ... after its named parameters. The arguments are words on the stack, so only types of one word (`int`, `unsigned`, pointers) can be read; a `char` is read as an `int`.

Macro	Does
`va_list`	The type of the variable that goes through the arguments.
`va_start(ap, last)`	Starts ap at the argument after *last*, the last named parameter.
`va_arg(ap, type)`	The next argument, of that type.
`va_end(ap)`	Ends the reading. It does nothing, but write it.

`IO.H`: Files by Handle

These functions are the services of COS for files (Chapter 7 of the COS User's Guide) without a buffer or the change of \n: what is written is what the file `gets`. A handle is a number; 0, 1 and 2 are the keyboard, the screen and the screen for errors. `IO.H` defines `O_RDONLY` (0), `O_WRONLY` (1), `O_RDWR` (2), `SEEK_SET` (0), `SEEK_CUR` (1), `SEEK_END` (2) and the variable `errno`. When a function fails it returns -1 and `errno` has the error code of COS (Appendix A of the COS User's Guide).

Function	Does
`int close(int fd)`	Closes a handle. 0, or -1.
`int creat(char *name, int attrib)`	Makes a file, or empties it, and opens it for writing. attrib is 0 for an ordinary file. Gives the handle.
`int isatty(int fd)`	1 if the handle is the keyboard or screen, 0 if it is a file.
`unsigned lseek(int fd, unsigned lo, unsigned hi, int whence)`	Moves the position of the file: to the offset lo + 65536 * hi counted from the start (`SEEK_SET`), the present position (`SEEK_CUR`) or the end (`SEEK_END`). Gives the low word of the new position.
`int open(char *name, int mode)`	Opens a file that exists, with `O_RDONLY`, `O_WRONLY` or `O_RDWR`. Gives the handle.
`int read(int fd, void *buf, unsigned n)`	Reads up to n bytes. Gives how many were read, 0 at the end of the file.
`int rename(char *old, char *new)`	Renames a file.
`int unlink(char *name)`	Deletes a file.
`int write(int fd, void *buf, unsigned n)`	Writes n bytes. Gives how many were written; less than n means the diskette is full.

DOS.H: The Machine

DOS.H defines struct REGS, with the unsigned members ax, bx, cx, dx, si, di and cflag, and the variable _psp, the segment of the program's PSP.

Function	Does
void disable(void)	Stops the interrupts of the processor (CLI). Do not leave them off for long.
void enable(void)	Allows interrupts again (STI).
unsigned getds(void)	The data segment of the program.
int inp(int port)	Reads a byte from a port.
unsigned inpw(int port)	Reads a word from a port.
int int86(int n, struct REGS *in, struct REGS *out)	Runs INT n with the registers of *in*; puts the registers it gives in *out*, and the carry flag in out->cflag. Gives AX.
int intdos(struct REGS *in, struct REGS *out)	int86 for INT 21h.
int outp(int port, int value)	Writes a byte to a port.
unsigned outpw(int port, unsigned value)	Writes a word to a port.
unsigned peek(unsigned seg, unsigned off)	The word at seg:off.
int peekb(unsigned seg, unsigned off)	The byte at seg:off, 0 to 255.
void poke(unsigned seg, unsigned off, unsigned value)	Writes a word at seg:off.
void pokeb(unsigned seg, unsigned off, int value)	Writes a byte at seg:off.
int spawn(char *path, char *args)	Runs a program, waits, and gives its return code, or -1 if it could not be run.

CONIO.H: Keyboard and Screen

Function	Does
int cputs(char *s)	Writes a string on the screen, as it is.
int getch(void)	Waits for a key and gives it, without showing it. Does not check Ctrl-C.

Function	Does
`int getche(void)`	Waits for a key, shows it and gives it.
`int kbhit(void)`	Not 0 if a key is waiting.
`int putch(int c)`	Writes a character on the screen, as it is.

This page intentionally left blank.

A

Appendix A. Error Messages

Contents

How Messages Look A-2
Messages About the Command Line and the Link A-2
Messages About Files and Lines A-3
Messages About the Preprocessor A-4
Messages About Declarations A-5
Messages About Expressions A-7
Messages About Statements A-8

How Messages Look

A mistake in a file is shown as *file*(*line*): *message*, where *file* is the name as Small-C opened it, a header file included. Other messages begin with SMALLC: and have no line. The names that Small-C shows are in italics here. When the program has several mistakes, correct the first and compile again: the others may be its consequence.

Messages About the Command Line and the Link

These begin with SMALLC: and have no file name.

Message	Meaning
`usage: SMALLC [options] file...`	Small-C was given no files, or an option it does not know, or -o with no name. The options are shown.
`SMALLC: cannot open` *`file`*	A file given to Small-C is not there (or, for a file with no extension, there is no .ASM of that name).
`SMALLC: cannot open the library` *`file`*	A library given to Small-C, or `CLIB.LIB`, cannot be read. `CLIB.LIB` must be next to `SMALLC.EXE`.
`SMALLC: cannot create` *`file`*	Small-C cannot write a file: the diskette is full or write-protected.
`SMALLC: cannot run` *`file`*	`ASM.COM` was not found next to `SMALLC.EXE` or in the root directory.
`SMALLC: too many files`	More than 24 files on the line.
`SMALLC:` *`file`* `would be written over`	A file given to Small-C has the name Small-C wants for `CCTEMP.ASM`, or for the .ASM of -S. Rename it.
`SMALLC: -c takes .C files, and .ASM files with -o:` *`file`*	With -c, a file that is not .C; .ASM files are only taken with -o.
`SMALLC: no` *`C0COM or C0EXE`* `start-up module in the libraries`	No library has the start-up module; with -n, give one with -L.
`SMALLC: '`*`name`*`' is not defined anywhere`	The program uses a name that no module or library defines.
`SMALLC:` *`name`* `is not defined anywhere`	The same, for a name of assembly language, without a C name.
`SMALLC:` *`name`* `is defined in two modules`	Two modules define the same function, or give the same variable a value (the name is shown with its underscore). A name may be defined once.

A

Message	Meaning
`SMALLC: the variables do not fit in 64 KB`	The variables of the program together are too large for a segment.
`SMALLC: too many errors`	Small-C stops after 20 messages.
`SMALLC: a library module has ;#def after ;#ref`	In a library, the `;#def` lines of a module must come before its `;#ref` lines.
`SMALLC: library too big`	A library has more than 600 names in all.
`SMALLC: too many library modules`	The libraries have more than 80 modules in all.
`SMALLC: too many variables in .bss`	More than 250 variables that start as zero in the whole program.

Messages About Files and Lines

Message	Meaning
`cannot open the include file` *name*	`#include` did not find the file in any of the places of Chapter 2.
`#include needs "file" or <file>`	The file name after `#include` is not in quotation marks or angle brackets.
`#include nested too deep`	Files include files more than 5 levels deep.
`line too long`	A line of more than 511 characters.
`line too long after its macros`	A line has more than 767 characters once its macros are replaced.
`comment without its end`	A /* comment was not closed before the end of the file.
`string too long`	A string of more than 299 characters. Write it as several strings next to each other.
`string without its end`	A string's closing quotation mark is missing on its line.
`bad character constant`	A character constant that is not one character, or lacks its closing '.
`bad number`	A number followed by letters, or with a point.
`long constants are not supported`	A number ending in L.
`unexpected character`	A character that is not part of C, such as @ or $.

Messages About the Preprocessor

Message	Meaning
`#define needs a name`	`#define` was not followed by a name.
`bad macro parameters`	The parameters of a macro are not names separated by commas, or there are more than 16.
`the # and ## operators are not supported`	A macro uses # or ##.
`wrong number of arguments for a macro`	A macro is called with more or fewer arguments than it has parameters.
`the arguments of a macro must be on one line`	The closing parenthesis of a macro call is not on its line.
`a macro expands without end`	A macro uses itself, directly or through others.
`bad #if expression`	The expression of `#if` or `#elif` is not made of numbers and operators.
`')' missing in #if`	A parenthesis of an `#if` expression is not closed.
`#else without #if`	`#else` or `#elif` with no `#if` before it.
`#endif without #if`	An `#endif` that closes nothing.
`#if without #endif`	An `#if` is open at the end of the file.
`#error` *text*	An `#error` line was reached; the text is what it said.
`unknown directive`	A line that begins with # and is not a directive SMALLC knows.
`#asm without #endasm`	The `#asm` block does not end before the end of the file.
`#asm block too long`	An `#asm` block of more than 767 characters.
`too many macros`	More than 250 macros, or more than 3000 characters of macros, in a source file.
`too many names`	More than 1250 different names, or 10000 characters of names.
`#if nested too deep`	More than 16 levels of `#if`.

A

Messages About Declarations

Message	Meaning
`long is not supported`	`long` was used.
`floating point is not supported`	`float` or `double` was used.
`typedef is not supported`	`typedef` was used.
`bit fields are not supported`	A member of a structure with a colon and a width.
`pointers to functions are not supported`	A declarator with parentheses, a pointer to a function, or the address of a function.
`structures are passed by pointer, not by value`	A structure as a parameter or as an argument.
`functions return structures by pointer, not by value`	A function declared to return a structure.
`a function cannot return an array`	A function declared to return an array.
`two storage classes`	For example `static extern`.
`two types`	For example `char int`.
`unsigned with a type that has no sign`	`unsigned` before a structure, a `void` or another type with no sign.
`auto or register outside a function`	Those classes are for variables of functions.
`a type expected`	A declaration at the top level has a name and no type, and is not a function.
`a name expected`	A declaration has no name where one is needed.
`a member type expected`	A structure member without a type.
`a parameter type expected`	A parameter without a type.
`a parameter declaration expected`	In an old-style definition, what follows the parentheses is not a declaration.
`a parameter without a name`	A function definition with a parameter that has only a type.
`not a parameter`	An old-style definition declares a name that is not among the parameters.
`too many parameters`	More than 32 parameters.
`a storage class in a type name`	`static` or `extern` in a cast or `sizeof`.
`a name in a type name`	A cast or `sizeof` (type) with a name in it.

Message	Meaning
`a structure needs a name or its members`	`struct` alone.
`a structure not yet defined`	A variable of a structure type that has no members yet.
`a member of a structure not yet defined`	A structure inside another before it is defined. Use a pointer, or define it first.
`struct '`*`name`*`' defined twice`	The members of a tag were written twice.
`member '`*`name`*`' twice`	Two members of a structure with the same name.
`'`*`name`*`' declared again`	A name that is an enumeration constant is declared again.
`'`*`name`*`' declared again as something else`	A name is declared once as a variable and again as a function (or the reverse).
`'`*`name`*`' defined twice`	A variable with an initial value twice.
`function '`*`name`*`' defined twice`	Two bodies for a function.
`static function '`*`name`*`' is not defined`	A `static` function that is used and has no body in the file.
`the size of '`*`name`*`' is not known`	A variable of an array with no size and no initial value, or an unfinished structure.
`the size of a local is not known`	The same, in a function.
`the size of a static is not known`	The same, for a `static` of a function.
`an array has from 1 to 32767 elements`	An array size that is 0, negative, or above 32767.
`only the first dimension can be left out`	A [] that is not the first of an array.
`an initializer must be a constant`	An initial value that is not a number, an address or a string.
`extern with an initializer`	An `extern` variable with an initial value.
`too many initializers`	More values than the array has elements.
`a union takes one initializer`	More than one value for a `union`.
`one string per pointer`	A pointer initialized with several strings.
`one string per array`	A `char` array initialized with several strings.
`the string is longer than the array`	A string that does not fit the `char` array it initializes.

A

Message	Meaning
`only simple locals can be initialized here; make it static`	An array or structure of a function with an initial value.
`too many dimensions`	More than 8 dimensions.
`too many globals`	More than 320 names of variables, functions and enumeration constants in a file.
`too many locals`	More than 100 names of variables and parameters in a function at one time.
`too many members`	More than 200 structure members in a file.
`too many structures`	More than 80 structures and unions in a file.
`too many types`	More than 200 different types in a file.
`too many strings in one initializer`	More than 40 strings, or 600 characters of strings, in the initial value of one variable.

Messages About Expressions

Message	Meaning
`'`*`name`*`' is not declared`	A name used and not declared. The most often it is a spelling mistake, or a missing `#include`.
`an expression expected`	An expression is not complete.
`a constant is needed`	A case, an array size or an enumeration value is not a constant expression.
`division by zero`	A division by a constant zero.
`an integer is needed here`	An operator that needs integers has a pointer or another type.
`a number or a pointer is needed here`	A structure or an array where a number is needed.
`an lvalue is needed for &`	& of something with no address.
`an lvalue is needed for =`	The left of = is not a variable.
`an lvalue is needed for an assignment`	The left of += and others is not a variable.
`++ or -- needs a variable`	++ or -- of an expression.
`only += and -= on pointers`	*=, &= and others on a pointer.
`* of something that is not a pointer`	* applied to a number.

Message	Meaning
`* of a void pointer`	* applied to a `void` *.
`arithmetic on a pointer to something without a size`	Adding to a `void` * or to a pointer to a function.
`sizeof of something without a size`	`sizeof` a function or `void`.
`only functions can be called`	A call with parentheses of something that is not a function.
`-> on something that is not a pointer to a structure`	The left of -> is not a pointer to a structure.
`. on something that is not a structure`	The left of . is not a structure.
`no member '`*`name`*`'`	The structure has no member with that name.
`casts are between numbers and pointers`	A cast to or from a structure or array.
`a value of the wrong type`	A structure or array where a number is needed.
`a void value used`	The value of a `void` function is used.
`structures of different types`	A structure assigned to another of a different type.
`expression too complex`	More than 250 parts in one expression.

Messages About Statements

Message	Meaning
`declarations go at the start of a block`	A declaration after a statement.
`a condition must be a number or a pointer`	The condition of if, while, do or for is a structure.
`a void function returns no value`	return with a value in a `void` function.
`break outside a loop or a switch`	break where there is nothing to leave.
`continue outside a loop`	continue where there is no loop.
`case outside a switch`	case not inside a switch.
`default outside a switch`	default not inside a switch.
`two defaults`	A switch has two default labels.
`the same case twice`	Two cases of a switch have the same value.
`switch needs an integer`	The expression of a switch is a pointer or another type.

Message	Meaning
`label 'name' twice`	Two labels with the same name in a function.
`label 'name' is not defined`	`goto` to a label that the function does not have.
`loops nested too deep`	More than 32 loops and switches nested.
`too many cases`	More than 150 cases in a function.
`too many labels`	More than 64 labels in a function.
`'c' expected`	The punctuation shown is missing: ;) } and so on. The message gives the line where Small-C found something else, often the line after the mistake.

This page intentionally left blank.

B

Appendix B. Technical Information

Contents

The Calling Convention B-2
Names and Labels B-2
Data B-3
Memory of a .COM Program B-3
Memory of an .EXE Program B-3
Limits of the Compiler B-4
Keywords B-4
A Module B-5

The Calling Convention

This is what to know to write functions in assembly language that C can call, or to call C from assembly language.

- The caller pushes the arguments, each as a word, from the *last* to the first, calls the function, and takes the arguments off the stack itself (ADD SP, n). A `char` argument is pushed as an `int`. At the start of the function, [sp] is the return address and [sp+2] the first argument.
- A function that uses its stack frame begins with PUSH BP and MOV BP, SP. Then the first parameter is at [bp+4], the second at [bp+6], and so on; the variables of the function are below BP, at [bp-2], [bp-4]...
- The value is returned in AX.
- BP, SI, DI, DS, ES and SS must be as they were when the function ends. AX, BX, CX and DX may be changed. The direction flag is clear, and must be left so.
- A function of C that is called with the wrong number of arguments cannot know it; the stack is put right by the caller, so it keeps its balance, but the function reads what is not there.

Names and Labels

Name	Is
`_name`	A function or variable of C: a leading underscore. Library functions follow this rule too.
`MODULE@name`	A `static` function or variable. MODULE is the name of the module, which is the name of its file in capitals.
`MODULE@V`*n*	A `static` variable of a function, that starts as zero. With an initial value it is .V*n*.
`.L`*n*`, .S`*n*`, .U`*name*`, .R, .F`	Local labels of a function: of jumps, of strings, of the labels of the program (`goto`), of the end of the function and of the size of its frame. They begin with a dot: they belong to the function, and each function has its own.
`__start, _exit, __exit, __psp, __brk, __brkend, __dbase, __image`	Names of the start-up modules.

Every address of data is written as *label* - `__dbase`: in a .COM program `__dbase` is 0 and the address is the offset; in an .EXE it is the start of the data segment, and the result is the offset in that segment. Code that is not made by Small-C must follow it for the same program to be made for both.

Data

- `int`, `unsigned`, pointers and enumerations are 2 bytes, low byte first. `char` is 1 byte.
- The members of a structure follow each other with no padding; a structure may begin at any address. The members of a `union` all begin at its start. An array of arrays is stored with the last index changing fastest.
- A string is a series of bytes with a zero after it, in the data of the program. The data that have an initial value come first, then the variables that start as zero, which are not in the file of the program, but placed after it when it starts, and set to zero.

Memory of a .COM Program

The program is loaded at offset 0x100 of a segment, after its PSP, and its code, its initialized data and its other variables follow, in that order. The program asks COS for 64 KB and gives back the rest of memory. The stack starts at the top of the segment. The heap goes from the end of the variables, up, to 2 KB below the stack pointer at the start.

Offset	Contents
`0000h`	The PSP
`0100h`	Code
`...`	Initialized data
`...`	Variables that start as zero
`...`	Heap, growing up
`...`	2 KB kept free
`...`	Stack, growing down
`FFFEh`	Top of the stack

Memory of an .EXE Program

An .EXE has a header with one relocation, which sets the data segment, and then its code. The code is one segment, up to 64 KB, as in a .COM. The data segment is another, after the code (on a boundary of 16 bytes): it holds the initialized data, the variables that start as zero, the heap, and the stack, which begins at the top of the segment, offset 0, wrapped, and goes down. The program asks for the memory it needs, at least the variables and 4 KB of stack, and gives back what it does not use of the rest. The heap stops 2 KB below the stack, as in a .COM. CS and DS are different segments, so the data may be the whole 64 KB, less the stack.

Limits of the Compiler

Item	Limit
Names	1250 different names (identifiers, macros and link names, in all the files of a compilation); their characters, 10000.
Globals	320 variables, functions and enumeration constants in a source file.
Locals	100 variables and parameters of a function that are in scope at once.
Types	200 different types (every pointer type and array type counts) in a source file.
Structures	80 structures and unions, and 200 members in all, in a source file.
Macros	250, with 3000 characters of text, in a source file. Parameters: 16.
Expression	250 parts.
Statements	150 cases in a function, 32 loops and switches nested, 64 labels in a function.
Declarations	32 parameters; 8 dimensions; 40 strings and 600 characters of them in one initializer.
Source	511 characters in a line, 767 after macros; 299 in a string; 767 in a `#asm` block; `#include` 5 deep; `#if` 16 deep; 20 errors.
Command line	24 files; 16 -D; 3 -I; 3 -L (and `CLIB.LIB`). A path of up to 79 characters.
Link	80 modules and 600 names in the libraries; 250 variables that start as zero.
Program	Code up to 64 KB; variables up to 64 KB, less the stack (and, in a .COM, less the code). One array or structure under 32 KB.
Arguments	24 words, 127 characters, for `argc` and `argv`.
Streams	7 open files at once.

Keywords

B

These words are reserved:

```
auto        else        register    switch
break       enum        return      union
case        extern      short       unsigned
char        for         signed      void
continue    goto        sizeof      while
default     if          static
do          int         struct
```

`const` and `volatile` are accepted and ignored. `long`, `float`, `double` and `typedef` are reserved, and give messages.

A Module

This is `TINY.C`, and `TINY.ASM`, the module that **smallc -c tiny.c** makes from it. The `;#bss` line is the `static` variable count, the `;#def` lines the two names that other files can use; bump is `static`, so it is TINY@bump.

```
/* TINY.C - a small module */
static int count;
char *label = "sum";

static int bump(void)
{
    return ++count;
}

int add(int a, int b)
{
    int t;

    t = a + b;
    bump();
    return t;
}
```

```
;#module TINY
section .data
_label:
    dw .S1-__dbase
.S1:
    db 'sum', 0
section .text
TINY@bump:
    push bp
    mov bp, sp
    times !!.F sub sp, .F
    inc word [TINY@count-__dbase]
    mov ax, [TINY@count-__dbase]
.R:
    times !!.F mov sp, bp
    pop bp
    ret
.F equ 0
_add:
    push bp
    mov bp, sp
    times !!.F sub sp, .F
    mov ax, [bp+4]
    add ax, [bp+6]
    mov [bp-2], ax
    call TINY@bump
    mov ax, [bp-2]
```

```
.R:
    times !!.F mov sp, bp
    pop bp
    ret
.F equ 2
;#bss TINY@count 2
;#def _label
;#def _add
```

C

Appendix C. The Sample Program WORDS.C

Contents

Counting Words C-2
The Listing C-2

Counting Words

WORDS counts the words of a text file and lists the commonest. It shows structures and tables of them, a `static` table, strings in memory from `strdup`, files, the command line, the macro that a -D can change, and a sort. Compile it with **smallc words.c**; with **smallc -DMAXWORDS=500 words.c** it has room for more different words.

```
A:\>smallc words.c
WORDS.COM: 7540 bytes

A:\>words sample.txt 8
72 words, 33 different
the            12  16%
is              8  11%
computer        6   8%
list            6   8%
a               5   6%
and             4   5%
small           2   2%
program         2   2%

A:\>words nofile.txt
WORDS: cannot open nofile.txt

A:\>
```

- find looks a word up and makes a new entry when it is new, returning a pointer to the entry, so `main` can count it with e->count++.
- getword reads letters from the file with `getc` and ctype's `isalpha`, and makes them small with `tolower`.
- sort uses structure assignment: a structure cannot be passed to a function, but may be copied.
- The percentage is worked out as an `unsigned` number, so that count * 100 does not overflow an `int`.

C

The Listing

```
/* WORDS.C - counts the words of a text file and lists the commonest.
 *
 *   WORDS file [count]
 *
 * A word is a run of letters (and apostrophes); capitals count as small
 * letters. Make it with: SMALLC WORDS.C   (or SMALLC -DMAXWORDS=500 WORDS.C)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <ctype.h>

#ifndef MAXWORDS
#define MAXWORDS 300            /* different words kept */
#endif
#define MAXLEN 20               /* longer words are cut */

struct entry {
    char *word;
    int count;
};

static struct entry table[MAXWORDS];
static int used;                /* entries of table in use */

/* The entry of a word, made if it is new; 0 when there is no room. */
static struct entry *find(char *w)
{
    int i;

    for (i = 0; i < used; i++)
        if (strcmp(table[i].word, w) == 0)
            return &table[i];
    if (used == MAXWORDS)
        return 0;
    table[used].word = strdup(w);
    if (table[used].word == 0)
        return 0;
    table[used].count = 0;
    return &table[used++];
}

/* Reads the next word of f into w: its length, or 0 at the end. */
static int getword(FILE *f, char *w)
{
    int c, n;

    do
        c = getc(f);
    while (c != EOF && !isalpha(c));
    n = 0;
    while (c != EOF && (isalpha(c) || c == '\'')) {
        if (n < MAXLEN)
            w[n++] = tolower(c);
        c = getc(f);
    }
    w[n] = 0;
    return n;
}

/* Puts the table in order of count, the commonest first. */
static void sort(void)
{
    int i, j;
    struct entry e;

    for (i = 1; i < used; i++) {
        e = table[i];
        for (j = i - 1; j >= 0 && table[j].count < e.count; j--)
            table[j + 1] = table[j];
```

C

```
        table[j + 1] = e;
    }
}

int main(int argc, char **argv)
{
    FILE *f;
    char w[MAXLEN + 1];
    struct entry *e;
    int i, top, total, lost;

    if (argc < 2) {
        printf("usage: WORDS file [count]\n");
        return 1;
    }
    f = fopen(argv[1], "r");
    if (f == 0) {
        printf("WORDS: cannot open %s\n", argv[1]);
        return 1;
    }
    top = argc > 2 ? atoi(argv[2]) : 10;
    total = 0;
    lost = 0;
    while (getword(f, w)) {
        total++;
        e = find(w);
        if (e)
            e->count++;
        else
            lost++;
    }
    fclose(f);
    sort();
    printf("%d words, %d different\n", total, used);
    for (i = 0; i < used && i < top; i++)
        printf("%-12s %4d %3u%%\n", table[i].word, table[i].count,
               (unsigned)table[i].count * 100 / total);
    if (lost)
        printf("%d words were not counted: the table is full\n", lost);
    return 0;
}
```

C

Quick Reference

Task	How
Compile and make a .COM	SMALLC *file*.C
Make an .EXE	SMALLC -e *file*.C
Name the program	SMALLC -o *name file*.C
Several files	SMALLC *a*.C *b*.C *c*.ASM *lib*.LIB
Define a macro	SMALLC -D*name=value* ...
Where the headers are	SMALLC -I*directory* ...
Keep the assembly language	SMALLC -S ...
A module for each file	SMALLC -c *a*.C *b*.C
A library	SMALLC -c -o *lib*.LIB *a*.C *b*.C *c*.ASM
Files and text	STDIO.H: fopen, fclose, getc, putc, fgets, fputs, printf, sprintf, fread, fwrite, remove, rename
Memory and numbers	STDLIB.H: malloc, calloc, realloc, free, exit, abs, atoi, itoa, rand, srand, getenv
Strings	STRING.H: strlen, strcpy, strcat, strcmp, strchr, strstr, strdup, memcpy, memset, memcmp...
Characters	CTYPE.H: isalpha, isdigit, isspace, toupper, tolower...
Handles	IO.H: open, creat, close, read, write, lseek, unlink
The machine	DOS.H: inp, outp, peek, poke, int86, intdos, spawn, disable, enable
Keys and screen	CONIO.H: getch, getche, kbhit, putch, cputs
Assembly language	#asm ... #endasm